

Programmazione Android

Fondamenti

Prof. Marco Camurri

Riferimenti

Il contenuto di questa presentazione è frutto della traduzione e "semplificazione" del materiale presente nella seguente guida ufficiale:

<https://developer.android.com/guide/components/fundamentals>

App components

- Ogni app Android è composta da "blocchi di base" detti *app components* (sono i "mattoni di base" con cui costruire le app):
 - ***activity***
 - ***service***
 - ***broadcast receiver***
 - ***content provider***
- Un'app non deve per forza usare tutti e quattro i tipi di componente (le app più semplici di solito sono composte solo Activity)
- A ciascun tipo corrisponde una classe Java (Activity, Service, BroadcastReceiver, ContentProvider) che viene *estesa* dal programmatore

Activity

- Il componente **activity** rappresenta una **singola schermata** dell'applicazione
- Più formalmente, l'activity è definita dal framework come una *singola attività svolta dall'utente per mezzo di un'interfaccia grafica*.
- Questa definizione sottolinea il fatto che ad ogni activity dovrebbe corrispondere una ben precisa "finalità" dal punto di vista dell'utente
- Esempi di activity:
 - Activity per comporre una nuova e-mail
 - Activity per visualizzare l'elenco delle email ricevute
 - Activity per scattare una foto

Service

- Il componente **service** rappresenta un blocco di codice eseguito in *background* (cioè in un thread secondario) e che continua ad essere eseguito anche dopo la chiusura dell'app
- Esempio 1:

Un'app di messaggistica vuole mostrare una notifica all'arrivo di un nuovo messaggio.

Occorre avviare un *service* che controlli periodicamente (ad esempio ogni secondo) la presenza di nuovi messaggi su un server. Il service dovrà essere avviato automaticamente all'accensione del dispositivo e dovrà rimanere sempre in esecuzione (anche quando l'app è chiusa!)

Service

- Esempio 2:

Un player musicale necessita di un service per riprodurre musica in *background* anche quando l'app non è in primo piano (*foreground*).

Broadcast Receiver

- I componenti di tipo ***broadcast receiver*** rappresentano degli oggetti “ascoltatori” in grado di reagire a determinati eventi generati dal S.O. o da altre app.
- Sono porzioni di codice della nostra app richiamabili automaticamente dal sistema operativo quando si verificano determinati eventi come:
 - arrivo di un SMS o di una telefonata
 - attivazione/disattivazione di WIFI, Bluetooth, 4G, modalità aereo, ...
 - scatto di una foto
 - completamento del boot (avvio del dispositivo)

Broadcast Receiver

- Con opportune istruzioni, l'app informa il S.O. che è interessata ad "ascoltare" determinati eventi e specifica il broadcast receiver che dovrà rispondere all'evento.
- Questa operazione è detta **registrazione** (**subscribe**) del broadcast receiver.
- Ogni volta che si verifica un evento, il S.O. esegue *tutti* * i broadcast receiver che si sono registrati come "ascoltatori" di quell'evento

* più app possono registrarsi per ascoltare lo stesso evento.

Content provider

- Un **content provider** è un componente attraverso cui un'app può fornire dati ad altre app (quindi agisce come "sorgente di dati")
- Se un'app implementa un content provider, le altre app possono interrogare (*to query*) il content provider per recuperare dei dati
- Un content provider può essere implementato anche direttamente dal S.O.
- L'esigenza di *definire* un content provider in un'app non è molto comune...
- ... mentre è frequente il caso di app che *utilizzano* un content provider implementato direttamente dal sistema.

Content provider di sistema

- Android implementa numerosi **content provider "di sistema"** che consentono alle app di accedere a contenuti gestiti dal sistema operativo. Esempi:
 - provider dei contatti in rubrica
 - provider degli eventi del calendario
 - provider delle raccolte multimediali (immagini, video, musica)

Content Provider

Esempio:

Un'app, dopo aver ottenuto i permessi necessari, può accedere all'elenco di tutti i numeri di telefono e nominativi presenti in rubrica.

Per farlo l'app dovrà interrogare il *content provider dei contatti*, fornito dal sistema.

Debug

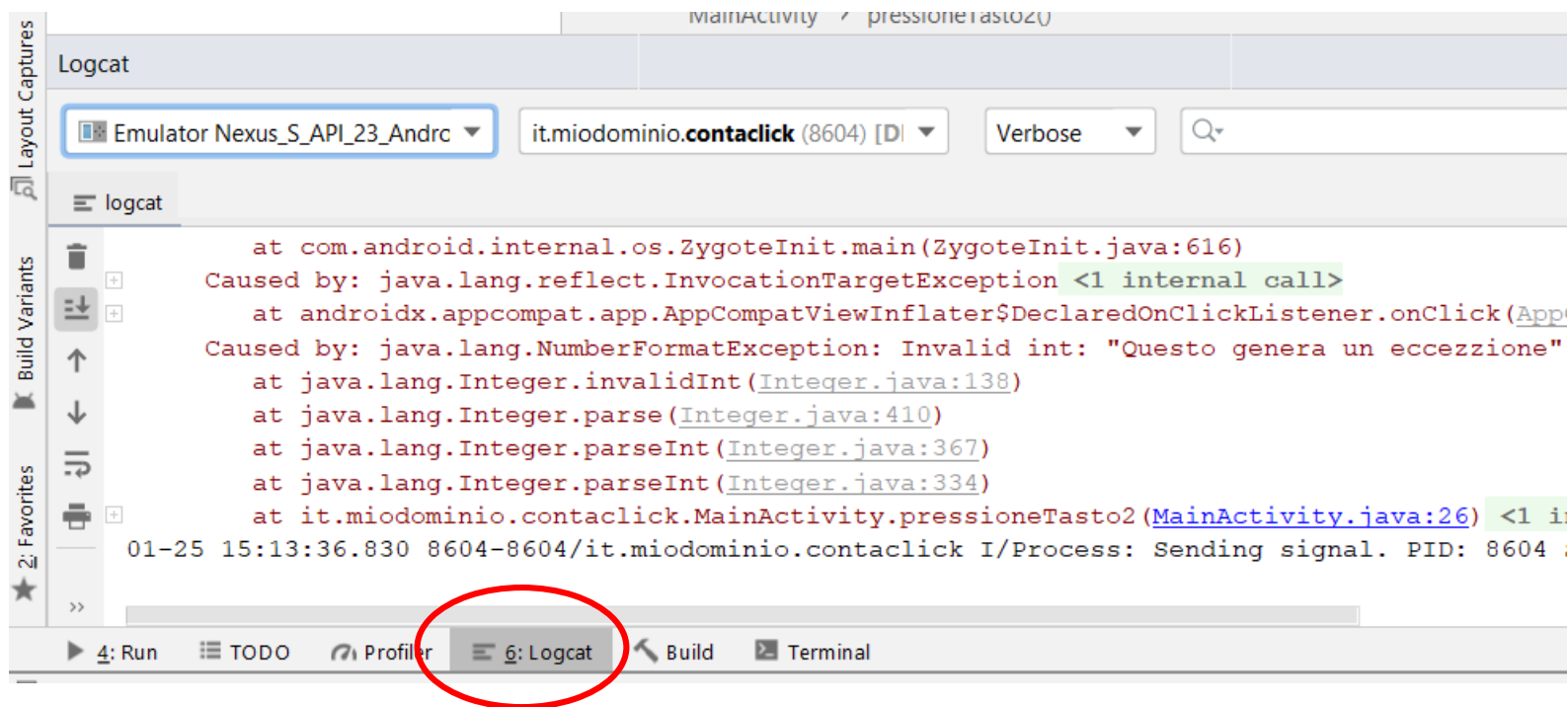
Debug dell'app con Logcat

- Se si verifica un'eccezione Java che non viene catturata, Android termina l'app
- Da Android Studio è possibile vedere la traccia dell'eccezione tramite la finestra Logcat.
- Esempio: proviamo a generare un'eccezione nel metodo pressioneTasto2

```
public void pressioneTasto2(View v) {  
    Integer.parseInt("Questo genera un eccezione");  
    //Intent intent = new Intent(this, SecondaActivity.class);  
    //startActivity(intent);  
}
```

Debug dell'app con Logcat

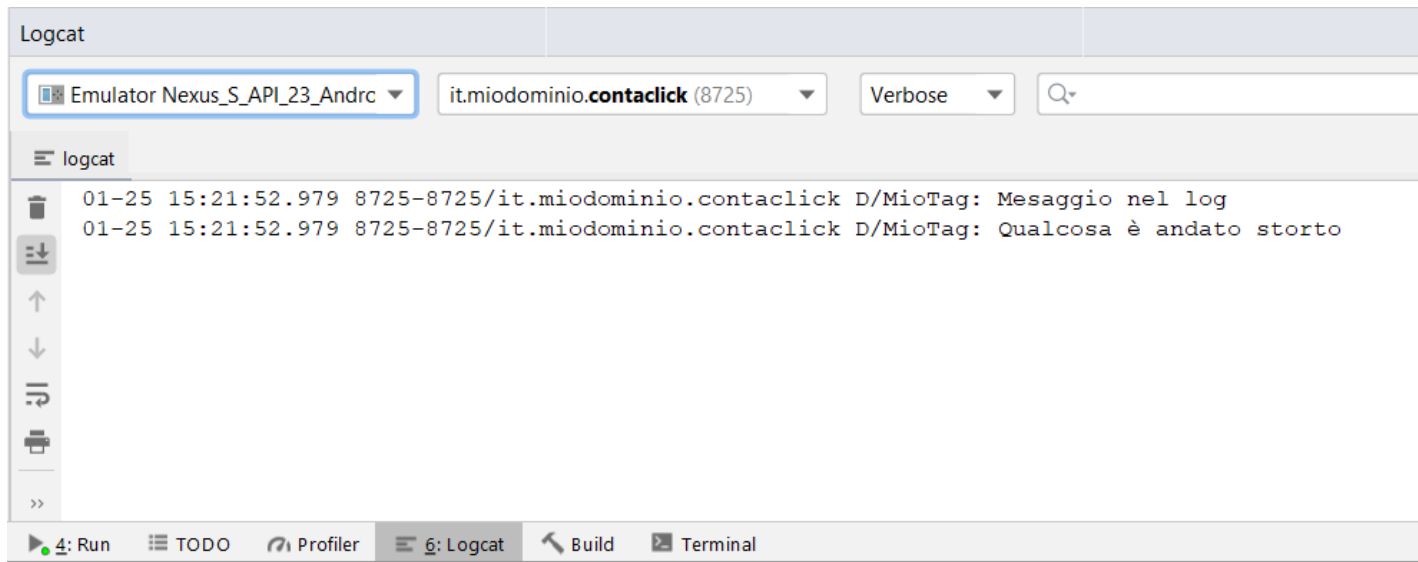
- Cliccando su Logcat (in basso) vedremo in rosso i dettagli dell'eccezione. L'ultima riga ci dice che l'eccezione si è originata alla riga 26 del file MainActivity.java



Debug dell'app con Logcat

- E' anche possibile inserire propri messaggi nel log:

```
public void pressioneTasto2(View v) {  
    Log.d("MioTag", "Messaggio nel log");  
    try {  
        Integer.parseInt("Questo genera un eccezione");  
    }  
    catch (Exception e) {  
        Log.d("MioTag", "Qualcosa è andato storto");  
    }  
}
```



Debug dell'app con Logcat

- La "d" del metodo **Log.d()** significa "livello di debug" (è possibile anche inserire messaggi di "livello error")
- Il primo parametro ("MioTag") è un'etichetta utile se si vogliono filtrare determinati messaggi nel log

Distribuzione

Distribuire l'app

Una volta terminata l'app è possibile distribuirla agli utenti usando uno dei seguenti metodi (dal più professionale al più "rudimentale")

- pubblicazione su uno *store* (come Google Play o altri)
- pubblicazione del *file APK* sul proprio sito web *
- trasferimento manuale del file APK dal PC al disco del dispositivo *

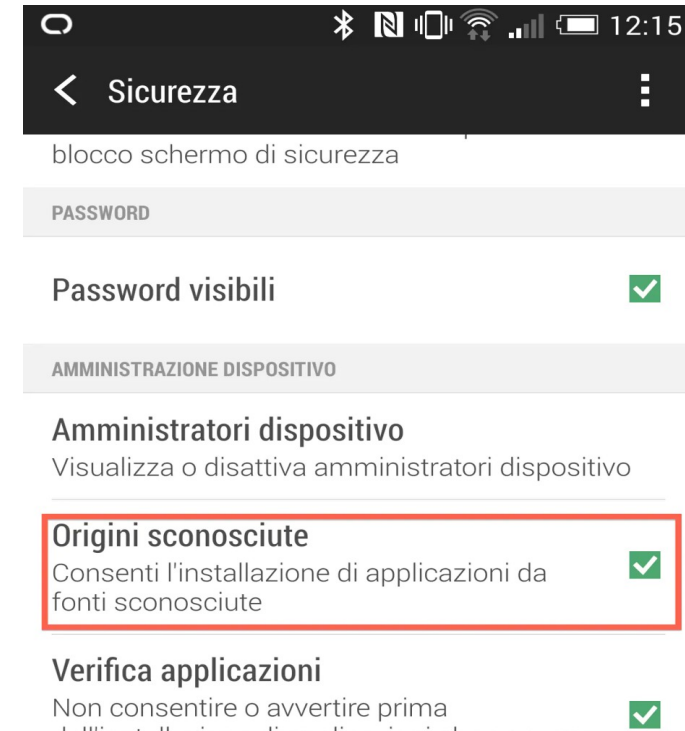
* per motivi di sicurezza questi due metodi richiedono che l'utente acconsenta all'installazione di app da "sorgenti sconosciute" (vedi slide successive)

File APK

- Il file **APK (Android Package)** è il file di installazione dell'app. Si tratta di una cartella compressa (zip) che contiene il file manifest, il codice compilato, e tutte le risorse necessarie all'app.
- In Android Studio si genera dal menu *Build* con l'opzione *Build APKs* (il file viene creato nella cartella `<radice_progetto>/app/build/outputs/apk/debug`)

Installazione da file APK

- Per proteggere gli utenti da possibili frodi Android consente l'installazione dell'app da file APK solo se è attivata l'opzione "*Origini sconosciute*" (o "*Sorgenti sconosciute*").



Esportare ed importare progetti

- Per **esportare** un progetto: **File | Export to Zip File...**

Nota bene: zippare manualmente la cartella del progetto non produce lo stesso effetto (si ottiene un file inutilmente più grande)

- Per **importare** un progetto:
 - decomprimere il file zip
 - scegliere *Open an existing Android Studio Project*
 - selezionare la cartella da aprire (cliccare sulla cartella su cui compare l'icona del compasso)

